



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2283 - DISEÑO Y ANÁLISIS DE ALGORITMOS

Ayudantía 8 - Programación Codiciosa

Octubre de 2025

Nicolás Buzeta, Manuel Villablanca

1. Soldadura Mínima

Una empresa se encarga de producir cañerías industriales de gran tamaño. Para formar una cañería de un cierto largo, el proceso avanza soldando pares de caños relativamente cortos. La máquina que realiza la soldadura debe tomar un par de caños que están en el suelo y elevarlos a cierta altura, para luego soldarlos. Todo esto consume una cierta cantidad de energía y combustible. Después de un tiempo de trabajo, los administradores de la empresa descubren que la cantidad de energía y combustible que necesitan para la producción es mayor mientras más pesados sean los caños que deber soldar, por lo que quieren optimizar el proceso. El peso de un caño depende de su longitud: mientras más largo, más pesado. Por lo tanto, dado un conjunto $C = \{c_1, \dots, c_n\}$ de n caños, en donde un caño c_i tiene longitud l_i , para $1 \leq i \leq n$, el objetivo es conectar **todos** esos caños para obtener una única cañería de longitud $L = \sum_{i=1}^n l_i$. El proceso lleva a cabo una secuencia de pasos, cada uno de los cuales realiza las siguientes acciones:

- Extrae dos caños del conjunto C , los cuales tienen longitud l_i y l_j .
 - Suelda los dos caños para producir un caño de longitud $l_i + l_j$.
 - Agrega el caño resultante al conjunto y recibe una penalización cuyo valor es $l_i + l_j$ (debido al peso que tiene que cargar la máquina).
- a) Piensa en un algoritmo codicioso. Solo se puede usar algoritmos de ordenamiento.
- b) Piensa en un algoritmo codicioso mas eficiente que use cualquier algoritmo.

1.1. Solución

a)

```
1 def greedy_brute(tubos: List[int]) -> int:
2     penalty = 0
3     while len(tubos) != 1:
4         tubos = sorted(tubos)
5         smallest = tubos[0]
6         second_smallest = tubos[1]
7
8         tubos = tubos[2:]
9         tubos.append(smallest + second_smallest)
10        penalty += smallest + second_smallest
11
12    return penalty
```

b)

```
1 def greedy(tubos: List[int]) -> int:
2     n_tubos = len(tubos)
3     heapq.heapify(tubos)
4
5     penalty = 0
6     while n_tubos != 1:
7         smallest = heapq.heappop(tubos)
8         second_smallest = heapq.heappop(tubos)
9
10        heapq.heappush(tubos, smallest + second_smallest)
11        penalty += smallest + second_smallest
12        n_tubos -= 1
13
14    return penalty
```

2. Intervalos Unitarios Enteros

Sea $A [1, \dots, n]$ un arreglo de números reales. Escriba un algoritmo greedy (usando pseudo-código o el lenguaje de su preferencia) que devuelva la cantidad mínima m de intervalos unitarios enteros que contengan a los elementos de A , con $1 \leq m \leq n$. Un intervalo unitario entero es de la forma $[x, x + 1]$, para $x \in \mathbb{N}$.

- Piensa en un algoritmo de fuerza bruta que resuelva el problema.
- Piensa en un algoritmo codicioso que resuelva este problema en $\mathcal{O}(n \log n)$. En caso de ser necesario, puede utilizar cualquier algoritmo conocido como subrutina.
- Demuestre que su algoritmo es óptimo.

2.1. Solución

a)

```

1 def check_cover(nums: List[int], intervals: Tuple[Tuple[int, int]]):
2     for num in nums:
3         covered = False
4         for interval in intervals:
5             if num >= interval[0] and num <= interval[1]:
6                 covered = True
7
8         if not covered:
9             return False
10
11     return True
12
13
14 def brute_force(nums: List[int]) -> int:
15     min_num = min(nums)
16     max_num = max(nums)
17
18     min_starting = math.floor(min_num)
19     max_starting = math.ceil(max_num)
20
21     possible_intervals = []
22
23     for x in range(min_starting, max_starting + 1):
24         interval = (x, x + 1)
25         possible_intervals.append(interval)
26
27     # between 0 and all intervals
28     n_possible_interval_sizes = len(possible_intervals) + 1
29
30     for interval_size in range(n_possible_interval_sizes):
31         for combination in itertools.combinations(possible_intervals, interval_size):
32             covered = check_cover(nums, combination)
33             if covered:
34                 return interval_size

```

```

35
36     raise ValueError("No possible interval cover")

```

b)

```

1 def greedy(nums: List[int]) -> int:
2     nums = sorted(nums) # Asumimos n log n
3
4     lower_bound_num = math.floor(nums[0])
5     intervals = [(lower_bound_num, lower_bound_num + 1)]
6
7     for num in nums:
8         lower_bound_num = math.floor(num)
9
10        # If in last interval, we don't add
11        last_interval = intervals[-1]
12        if num >= last_interval[0] and num <= last_interval[1]:
13            continue
14
15        intervals.append((lower_bound_num, lower_bound_num + 1))
16
17    return len(intervals)

```

- c) Vamos a demostrar que el algoritmo encuentra la mínima cantidad de intervalos unitarios tal que se cubren todos los valores entregados. Vamos a suponer que nuestro algoritmo entrega una solución correcta y ordenada (cubre todos los valores y los intervalos están ordenados).

Consideramos nuestra solución $I_1, I_2, \dots, I_p$. También consideramos una solución óptima cualquiera $J_1, J_2, \dots, J_q$, la cual también está ordenada.

Primero demostraremos que para todo $1 \leq k \leq \min\{p, q\}$, los primeros k intervalos $I_1, \dots, I_k$ cubren por lo menos tantos valores de A como los intervalos $J_1, \dots, J_k$.

Por inducción:

- **Caso Base:** ($k = 1$)

Por construcción de nuestro intervalo, podemos ver que I_1 corresponde al intervalo más alto posible que contiene el primer valor de A . Por lo que I_1 y J_1 deben ser iguales. Por lo que I_1 no puede cubrir menos valores que J_1 .

- **Hipótesis Inductiva:**

Supongamos que los intervalos $I_1, \dots, I_n$ cubren por lo menos tantos valores A que los intervalos $J_1, \dots, J_n$.

- **Paso Inductivo:**

Consideramos el primer valor de A cubierto por I_{n+1} , el cual llamaremos x . También consideramos el primer valor de A cubierto por J_{n+1} , el cual llamaremos y .

Basándonos en la hipótesis inductiva, $I_1, \dots, I_n$ debe cubrir por lo menos tantos valores de A como $J_1, \dots, J_n$. Dado esto $I_n \geq J_n$. Por construcción de nuestro intervalo, sabemos que $I_{n+1} > I_n$. Juntando esto, tenemos $I_{n+1} \geq J_{n+1}$. Como I_{n+1} es mayor o igual, I está ordenado, y I abarca todos los valores de A , $I_1, \dots, I_{n+1}$ cubre por lo menos tantos valores de A como $J_1, \dots, J_{n+1}$.

Ahora demostramos que nuestro algoritmo es óptimo. Por contradicción supongamos que $I_1, \dots, I_p$ no es óptimo, mientras que $J_1, \dots, J_q$ sí lo es.

Como I no es óptimo y J sí lo es, debemos tener que $q < p$. Por lo anterior, debe existir $I_1, \dots, I_q$. Por la demostración anterior, sabemos que $I_1, \dots, I_q$ debe cubrir por lo menos tantos valores de A como los intervalos $J_1, \dots, J_q$. Por lo que $I_1, \dots, I_q$ debe cubrir todos los valores de A si $J_1, \dots, J_q$ lo hace. Pero, por construcción, sabemos que los intervalos $I_{q+1}, \dots, I_p$ solo se agregan cuando faltan un valor por cubrir por el último intervalo agregado. Esto significa que no puede existir ninguno de estos intervalos.

Por lo que I debe ser óptimo. ■

Esta es básicamente la pregunta 5 de la guía. Sin embargo, esa pide los intervalos no necesariamente sean enteros. Pueden revisar la guía para comparar.

3. Eliminar Intervalos Redundantes

Sea $\mathcal{E} = \{(a_1, b_1], (a_2, b_2], \dots, (a_n, b_n]\}$ un conjunto de intervalos sobre la línea de los reales. Se quiere calcular un conjunto $\mathcal{R} \subseteq \mathcal{E}$ que tenga el menor tamaño posible y para el que se cumpla que $\text{cob}(\mathcal{E}) = \text{cob}(\mathcal{R})$, en donde definimos $\text{cob}(\mathcal{E}) = \{x \in \mathcal{R} \mid \exists (a, b] \in \mathcal{E}, x \in (a, b]\}$. De forma análoga se define $\text{cob}(\mathcal{R})$.

- Piensa en un algoritmo codicioso que resuelva este problema. En caso de ser necesario, puede utilizar cualquier algoritmo conocido como subrutina.
- Indique el tiempo de ejecución de su algoritmo, en función de la cantidad de intervalos en el conjunto de entrada.
- Demuestre que su algoritmo es optimo.

3.1. Solución

a)

```

1 def greedy(intervals: Set[Tuple[int, int]]) -> Set[Tuple[int, int]]:
2     sorted_intervals: List[Tuple[int, int]] = sorted(
3         intervals, key=lambda interval: (interval[0])
4     )
5
6     min_intervals: List[Tuple[int, int]] = []
7
8     s: int = sorted_intervals[0][
9         0
10    ] # everything to the left is already covered by min_intervals, including s
11
12    i = 0
13    while True:
14        intervals_s = [] # All intervals that contain s
15
16        # Add all intervals that are continuous already added min_intervals (start before or at s
17        while i < len(sorted_intervals) and sorted_intervals[i][0] <= s:
18            # Skip interval if it's already covered by min_intervals
19            if (
20                len(min_intervals) != 0
21                and sorted_intervals[i][1] <= min_intervals[-1][1]
22            ):
23                i += 1
24                continue
25
26            intervals_s.append(sorted_intervals[i])
27            i += 1
28
29        # No intervals that cover s
30        if len(intervals_s) == 0:
31            if i >= len(sorted_intervals): # No more intervals to check
32                break
33            s = sorted_intervals[i][0]
```

```

34     else:
35         max_left_interval = max(intervals_s, key=lambda interval: interval[1])
36         s = max_left_interval[1]
37         min_intervals.append(max_left_interval)
38
39     return set(min_intervals)

```

- b) El tiempo de ejecución es $\mathcal{O}(n \log n)$ por el paso de ordenamiento. Todo el proceso que hace luego de ordenar toma tiempo $\mathcal{O}(n)$, ya que se itera por todos los intervalos y se hacen operaciones de tiempo constante por cada uno de los intervalos.
- c) Vamos a demostrar que el algoritmo encuentra un subconjunto $R \in \mathcal{E}$, tal que R es del menor tamaño posible, tal que $\mathbf{cob}(R) = \mathbf{cob}(\mathcal{E})$. Vamos a suponer que nuestro algoritmo entrega una solución correcta y ordenada por el primer índice.

Consideramos nuestra solución $R_1, R_2, \dots, R_p$. También consideramos una solución óptima cualquiera $J_1, J_2, \dots, J_q$, la cual también está ordenada de la misma manera.

Primero demostraremos que para todo $1 \leq k \leq \min\{p, q\}$, $\bigcup_{i=1}^k J_i \subseteq \bigcup_{i=1}^k R_i$. Es decir, la unión de los primeros k intervalos de J no pueden cubrir más que los del conjunto R .

Por inducción:

■ **Caso Base:** ($k = 1$)

Por construcción de nuestro intervalo, podemos ver que R_1 corresponde al intervalo cuyo extremo izquierdo es a_1 (primer elemento) y cuyo extremo derecho es el intervalo con el extremo derecho más alto. Como hay que incluir a_1 y trivialmente es óptimo agarrar el intervalo más grande en esta situación, es claro que $J_1 \subseteq R_1$.

■ **Hipótesis Inductiva:**

Supongamos que la unión de los primeros n intervalos de cada solución cumplen lo que queremos demostrar. Es decir, $\bigcup_{i=1}^n J_i \subseteq \bigcup_{i=1}^n R_i$.

Sea s_n el valor de s en nuestro algoritmo después de haber agregado n intervalos a la respuesta. Sea $(a_n, b_n]$ el n -ésimo intervalo agregado, entonces $s_n \leq b_n$.

Sea t_n el valor extrema de la derecha más grande contenido en J_n

Por lo anterior tenemos que $t_n \leq b_n \leq s_n$

■ **Paso Inductivo:**

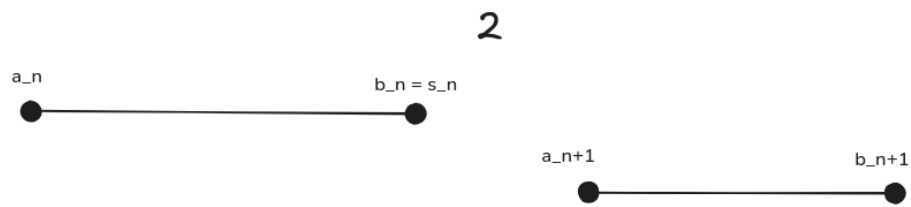
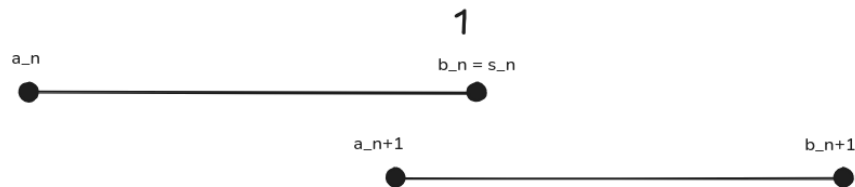
Sea:

- $(a_n, b_n]$ el n -ésimo intervalo agregado en R .
- $(a'_n, b'_n]$ el n -ésimo intervalo agregado en J .

Tal como se puede ver en el algoritmo, el intervalo $(a_{n+1}, b_{n+1}]$ debe cumplir:

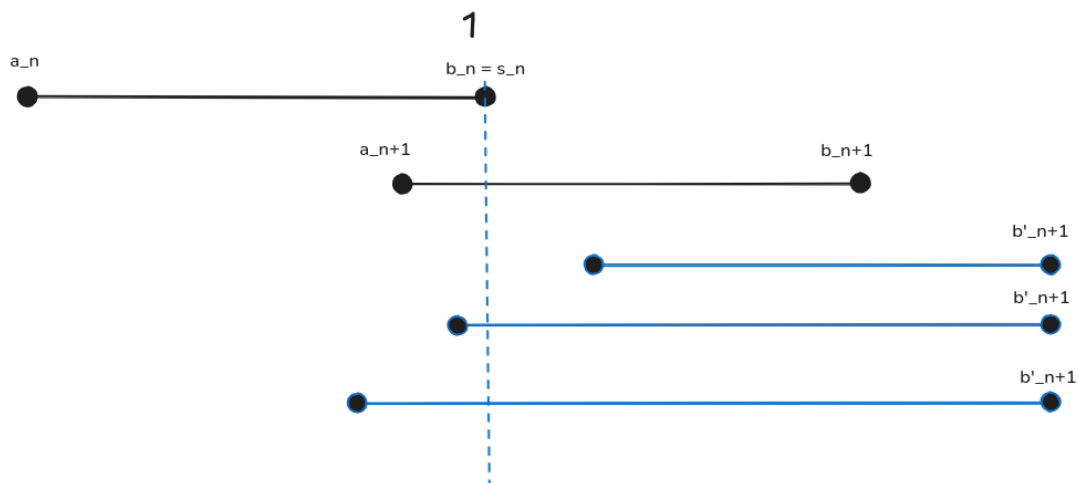
$$a_{n+1} \leq s_n \leq b_{n+1}$$

Tenemos dos casos al agregar: y



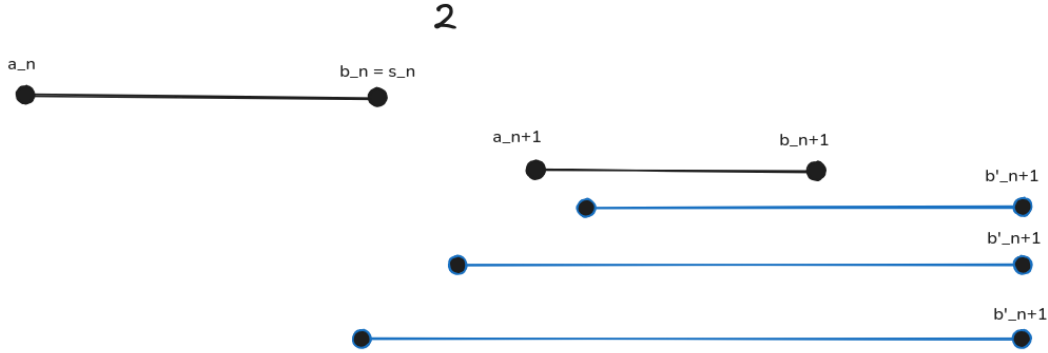
Ahora debemos mostrar que en ambos casos no puede ocurrir que $J_1, \dots, J_{n+1}$ vaya ‘adelante’ de $R_1, \dots, R_{n+1}$. Es decir, que cubra más con $n + 1$ intervalos.

- Para el caso 1, donde $a_{n+1} \leq s_n$, tenemos los siguientes casos:



Acá podemos ver 3 opciones. Vamos a decir A, B, C en orden que aparecen. En el caso de A, esta no puede ocurrir, ya que quedarían los elementos entre b_n y b'_n sin cubrir. El caso B y C no pueden ocurrir, ya que hubiese sido elegidos por el algoritmo enés de $(a_{n+1}, b_{n+1}]$

- Para el caso 2, donde $a_{n+1} > s_n$, tenemos los siguientes casos:



Por las mismas razones de antes, ninguno de estos casos pueden existir.

Ahora demostramos que nuestro algoritmo es óptimo. Por contradicción supongamos que $R_1, \dots, R_p$ no es óptimo, mientras que $J_1, \dots, J_q$ sí lo es.

Dado que $|J| = q$ y por la inducción, $\bigcup_{i=1}^q J_i \subseteq \bigcup_{i=1}^q R_i$. Es decir, $\bigcup_{i=1}^q J_i$ no puede cubrir más valores que $\bigcup_{i=1}^q R_i$.

Sin embargo, esto significa que los intervalos restantes de R no son necesarios para la solución. Por construcción, solo agregamos intervalos a R si son necesarios para cubrir todo el rango. Por lo que llegamos a una contradicción.

Por lo que R debe ser óptimo. ■